Documento de Justificación de Diseño Orientado a Objetos

Sistema EcoHydro-4 — Módulo Hidropónico Automatizado

Curso IC-2101 Programación Orientada a Objetos — ITCR Campus San Carlos

---

1. Introducción

El presente documento expone y justifica las decisiones de diseño adoptadas en la fase de modelado del sistema EcoHydro-4, un módulo hidropónico automatizado desarrollado en el marco del curso IC-2101 Programación Orientada a Objetos del Instituto Tecnológico de Costa Rica. Se abordan dos aspectos fundamentales: la abstracción de las cinco clases que conforman el dominio del sistema, y la naturaleza de las relaciones estructurales que las vinculan según los criterios de UML 2.5. El análisis se basa en los principios de encapsulamiento, responsabilidad única y gestión del ciclo de vida de los objetos.

---

2. Justificación de las clases del dominio

2.1 UnidadCentralControl

Esta clase constituye el núcleo operativo del sistema. Su abstracción se justifica porque concentra las dos reglas de negocio más críticas: el diagnóstico de viabilidad energética (RF1) y la ejecución del ciclo de muestreo hidropónico (RF2). Delegar estas responsabilidades en otra clase violaría el principio de responsabilidad única, mientras que integrarlas en una clase subordinada como `BrazoRobotico` introduciría un acoplamiento inadecuado entre el actuador físico y la lógica de control. Su existencia como clase independiente también permite modelar con precisión el ciclo de vida del sistema completo: la unidad es el “todo” del que dependen sus partes compuestas.

2.2 BrazoRobotico

El brazo robótico constituye el actuador físico del sistema hidropónico. Su abstracción responde a la necesidad de encapsular tres parámetros cinemáticos propios —velocidad de desplazamiento, ángulo de extensión y horas de uso acumuladas— junto con la lógica de validación asociada a cada uno. Incorporar estos atributos directamente en `UnidadCentralControl` produciría una clase con demasiadas responsabilidades (*God Object*). La independencia de `BrazoRobotico` como clase permite, además, modelar con exactitud la relación de Composición: el brazo existe única y exclusivamente en el contexto de la unidad que lo instancia.

2.3 BitacoraHistorica

La bitácora centraliza la gestión del historial de muestreos y toda la lógica de análisis químico definida en RF3: detección de anomalías, cálculo de porcentajes y generación de resúmenes. Su separación de `UnidadCentralControl` obedece al principio de responsabilidad única: la unidad controla el hardware; la bitácora administra los datos históricos. Esta separación tiene además una consecuencia de diseño fundamental: la bitácora debe sobrevivir a la destrucción de la unidad para preservar el historial de operación, lo que la convierte en candidata natural a una relación de Agregación.

2.4 ReporteMuestreo

Un reporte de muestreo es un objeto de valor (*value object*) que agrupa cuatro datos que pierden significado si se separan: la marca temporal, el pH, la conductividad eléctrica y el

gasto energético del ciclo. Su abstracción como clase independiente, sin setters, garantiza la inmutabilidad del registro una vez creado, lo que es esencial para la integridad del historial. La ausencia de métodos modificadores también refleja la naturaleza del dato: una medición tomada en un instante no puede alterarse retroactivamente.

2.5 Operador

El operador modela al ser humano responsable del sistema. Su abstracción se justifica porque posee identidad propia —ficha de empleado, nombre completo y correo electrónico— que existe con independencia de cualquier unidad hidropónica. Además, cumple un rol funcional concreto en RF5: es el destinatario de las alertas por anomalía química. Representarlo como una simple cadena de texto dentro de `UnidadCentralControl` impediría validar su correo, encapsular su información y eventualmente extender su comportamiento sin modificar otras clases.

---

3. Justificación de las relaciones estructurales

3.1 UnidadCentralControl → BrazoRobotico: Composición

La relación de Composición (rombo relleno ◆) se eligió porque el `BrazoRobotico` carece de existencia fuera de la `UnidadCentralControl` que lo contiene. Esta dependencia total del ciclo de vida se implementa instanciando el brazo **dentro del constructor** de la unidad, sin exponer ninguna referencia externa:

```
this.brazoRobotico = new BrazoRobotico(velBrazo, anguloBrazo, 0.0);
```

Cuando la unidad es destruida mediante `destruir()`, el campo se anula y el objeto queda inalcanzable para el recolector de basura. Ningún código externo puede recuperar el brazo porque nunca se publicó una referencia a él. Este es el criterio definitorio de la Composición en UML 2.5: la destrucción del todo implica la destrucción irrecuperable de la parte.

Utilizar Agregación en su lugar habría sido incorrecto: implicaría que el brazo puede existir y ser reutilizado sin la unidad, lo que contradice tanto el modelo de dominio como la implementación.

3.2 UnidadCentralControl → BitacoraHistorica: Agregación

La relación de Agregación (rombo vacío ◇) modela el vínculo todo-parte cuando la parte puede sobrevivir al todo. La `BitacoraHistorica` se crea antes que la unidad y se le inyecta como referencia externa:

```
BitacoraHistorica bitacora = new BitacoraHistorica("BIT-" + codigo);
UnidadCentralControl unidad = new UnidadCentralControl(..., bitacora, ...);
```

Al destruirse la unidad, la referencia `bitacora` en el llamador permanece válida y los reportes históricos se conservan intactos. Este comportamiento no es accidental: refleja el requisito del dominio de que los registros de muestreo deben estar disponibles para auditorías incluso si el hardware falla. La prueba de identidad de objeto en la simulación de falla confirma que el mismo objeto en memoria persiste antes y después de la destrucción.

3.3 UnidadCentralControl → Operador: Asociación

La Asociación simple (línea sólida sin rombo) expresa un vínculo de conocimiento sin implicaciones sobre el ciclo de vida. El `Operador` es una persona cuya existencia es completamente independiente del sistema hidropónico: puede estar asignado a múltiples unidades, puede no estar asignado a ninguna (`setOperador(null)` es válido), y su ciclo de vida no guarda relación con el de la unidad. La unidad simplemente *conoce* al operador para poder notificarle en caso de anomalía; no lo posee ni controla su existencia. Este vínculo de conocimiento sin control del ciclo de vida define exactamente la Asociación en UML 2.5.

3.4 BrazoRobotico → ReporteMuestreo: Dependencia

La Dependencia (línea punteada con flecha abierta) es la relación más débil del modelo y refleja un uso temporal de una clase por parte de otra. En Java se implementa mediante un

parámetro o variable local de método, nunca como atributo de instancia. En EcoHydro-4, el `ReporteMuestreo` se crea como variable local dentro de `ejecutarCicloMuestreo()` y se retorna al llamador:

```
        ReporteMuestreo reporte = new ReporteMuestreo(
                LocalDateTime.now(), ph, conductividad, gastoW);
        return reporte;
```

El brazo no retiene ninguna referencia al reporte tras retornar del método. Si se hubiera almacenado como atributo (`private ReporteMuestreo ultimo`), la relación correcta sería Asociación. La Dependencia documenta explícitamente que el acoplamiento es mínimo e intencional.

3.5 BitacoraHistorica → ReporteMuestreo: Agregación

La bitácora mantiene una colección de reportes (`ArrayList<ReporteMuestreo>`) cuyos objetos fueron creados externamente y se inyectan mediante `registrarReporte(reporte)`. La bitácora es el “todo” y los reportes son las “partes”, pero estas partes pueden existir fuera de la colección: el mismo objeto podría ser referenciado por el llamador antes y después de ser agregado. La cardinalidad 0..* refleja que una bitácora puede estar vacía al iniciar operaciones. Se eligió Agregación sobre Composición porque la bitácora no crea los reportes: los recibe ya construidos desde el ciclo de muestreo, lo que implica independencia de creación y, por tanto, potencial independencia de destrucción.

---

4. Conclusión

El modelo de dominio de EcoHydro-4 demuestra cómo los distintos tipos de relación estructural de UML 2.5 permiten expresar con precisión las dependencias de ciclo de vida entre objetos. La Composición garantiza que el brazo robótico no puede operar sin su unidad de control; la Agregación preserva la bitácora histórica y sus reportes ante fallos del hardware; la Asociación reconoce la autonomía del operador como entidad del dominio; y la Dependencia minimiza el acoplamiento en la generación de reportes, produciendo objetos cuyo ciclo de vida es gestionado exclusivamente por el llamador. En conjunto, estas decisiones de diseño favorecen la cohesión, reducen el acoplamiento y facilitan la mantenibilidad del sistema.

Documento elaborado para IC-2101 Programación Orientada a Objetos. Instituto Tecnológico de Costa Rica — Campus San Carlos.